

git

Test de repaso — parte 3: repositorios remotos

81 preguntas · 4 opciones, 1 correcta

Cómo usar este banco de preguntas

- Cada diapositiva contiene una pregunta con 4 opciones (A-D), solo 1 correcta.
- No incluye las respuestas: úsalo para practicar de cara al examen. TODAS las preguntas del examen saldrán de este banco de preguntas.

Bloque 1

Repositorio remoto y GitHub

Pregunta 1

¿Qué es un repositorio remoto?

- A) Una copia de tu repositorio en un servidor, accesible por Internet
- B) Un fichero de configuración de git
- C) La papelera de reciclaje de git
- D) Un tipo de commit especial

Pregunta 2

¿Qué es GitHub?

- A) Un lenguaje de programación
- B) Una plataforma para gestionar repositorios remotos
- C) Un comando de git
- D) Un tipo de licencia de software

Pregunta 3

¿GitHub es la única plataforma que existe para gestionar repositorios remotos?

- A) Sí, es la única
- B) No, también existen alternativas como GitLab o Bitbucket
- C) No, pero solo funciona con SVN
- D) Sí, es un estándar oficial de git

Pregunta 4

¿Qué permite un repositorio remoto que un repositorio solo local no permite?

- A) Hacer commits
- B) Que múltiples colaboradores contribuyan al proyecto a través de Internet
- C) Usar `.gitignore`
- D) Ver el historial con `git log`

Pregunta 5

Para crear un repositorio nuevo en GitHub desde la web, ¿qué botón usas?

- A) Add
- B) New
- C) Create --remote
- D) Push

Pregunta 6

Al crear un repositorio en GitHub, ¿con qué ficheros puedes inicializarlo opcionalmente?

- A) `index.html` y `styles.css`
- B) `README.md` o `.gitignore`
- C) `package.json` obligatoriamente
- D) Ningún fichero, siempre se crea vacío

Bloque 2

git clone y crear proyecto en GitHub

Pregunta 7

¿Qué hace `git clone <url>` ?

- A) Crea una rama nueva
- B) Descarga una copia completa de un repositorio remoto y configura ese remoto como `origin`
- C) Sube tus cambios locales al remoto
- D) Elimina el repositorio remoto

Pregunta 8

Después de hacer `git clone`, ¿hace falta ejecutar `git init`?

- A) Sí, siempre
- B) No, `git clone` ya deja el repositorio listo para trabajar
- C) Solo si el repositorio remoto está vacío
- D) Solo en Windows

Pregunta 9

Tienes un proyecto que ya tenías en local (con commits) y quieres subirlo por primera vez a un repositorio vacío en GitHub. ¿Qué secuencia de comandos usarías?

- A) `git clone` y luego `git commit`
- B) `git remote add origin <url>` y `git push -u origin main`
- C) `git pull origin main` y `git init`
- D) `git fetch` y `git merge`

Pregunta 10

Si vas a subir por primera vez a GitHub un proyecto que ya tiene commits en local, ¿por qué es buena idea **no** inicializar el repositorio remoto con `README.md` ?

- A) Porque GitHub no lo permite
- B) Porque las dos historias (local y remota) podrían no encajar directamente al hacer push
- C) Porque el README ocupa demasiado espacio
- D) No hay ningún motivo, es indiferente

Pregunta 11

¿Qué comando descarga un repositorio remoto entero, incluyendo su historial y ramas, a tu máquina?

- A) `git pull`
- B) `git fetch`
- C) `git clone`
- D) `git remote`

Pregunta 12

¿Qué remoto se configura automáticamente al hacer `git clone` ?

- A) `upstream`
- B) `origin`
- C) `main`
- D) Ninguno, hay que añadirlo a mano

Bloque 3

Autenticación con GitHub

Pregunta 13

¿GitHub sigue aceptando usuario y contraseña para hacer `git push` por HTTPS?

- A) Sí, siempre ha funcionado así y sigue igual
- B) No, desde 2021 ya no acepta contraseña; hace falta un token o SSH
- C) Solo los fines de semana
- D) Solo si el repositorio es privado

Pregunta 14

¿Qué es un PAT en el contexto de GitHub?

- A) Un tipo de rama
- B) Un token de acceso personal que se usa en vez de la contraseña
- C) Un comando de git
- D) El nombre del repositorio principal

Pregunta 15

¿Dónde se genera un token de acceso personal (PAT) en GitHub?

- A) En `git config --global`
- B) En Settings → Developer settings → Personal access tokens
- C) En el fichero `.gitignore`
- D) No se puede generar, viene por defecto

Pregunta 16

¿Qué alternativa a los tokens existe para autenticarte con GitHub por línea de comandos?

- A) Claves SSH
- B) `.gitignore`
- C) `git log --auth`
- D) Cambiar el nombre de la rama a `main`

Pregunta 17

Si usas autenticación por SSH en vez de HTTPS, ¿cómo cambia la URL del remoto?

- A) No cambia nada
- B) Usa un formato tipo `git@github.com:usuario/repo.git` en vez de `https://...`
- C) Deja de necesitar `git remote add`
- D) Solo funciona con `git clone`, nunca con `git push`

Bloque 4

Fork

Pregunta 18

¿Qué es un fork en GitHub?

- A) Una rama local
- B) Una copia de un repositorio que se crea en tu propia cuenta de GitHub, a partir de un proyecto de otro usuario
- C) Un comando de git
- D) Un tipo de commit

Pregunta 19

¿Para qué sirve hacer un fork antes de contribuir a un proyecto de código abierto?

- A) Para borrar el proyecto original
- B) Para hacer tus modificaciones sin tocar el código original, y luego proponerlas con un pull request
- C) Para evitar tener que hacer commits
- D) Para cambiar el dueño del repositorio original

Pregunta 20

Después de hacer fork de un repositorio, ¿en qué cuenta queda la copia?

- A) En la cuenta del autor original
- B) En tu propia cuenta de GitHub
- C) En ninguna cuenta, queda anónima
- D) Solo en tu máquina local

Pregunta 21

¿Cuál de estos NO es un motivo típico para hacer un fork, según lo visto en clase?

- A) Contribuir a proyectos de terceros
- B) Personalizar un proyecto sin compartir los cambios
- C) Explorar y experimentar sin riesgo para el proyecto original
- D) Borrar el historial del repositorio original

Pregunta 22

En el flujo típico de trabajo con un fork, después de clonarlo y modificarlo en local, ¿qué haces para subir tus cambios a tu copia en GitHub?

- A) `git pull`
- B) `git push`
- C) `git fetch`
- D) `git remote remove`

Pregunta 23

Después de hacer `push` a tu fork, ¿cómo propones tus cambios al repositorio original?

- A) Haciendo otro fork
- B) Abriendo un Pull Request desde tu fork hacia el repositorio original
- C) Borrando tu fork
- D) Enviando un email al autor

Pregunta 24

¿Quién revisa y decide si se fusionan los cambios propuestos en un Pull Request hecho desde un fork?

- A) Se fusionan automáticamente sin revisión
- B) Los mantenedores del repositorio original
- C) GitHub lo decide con inteligencia artificial
- D) Nadie, los PR no se pueden fusionar desde un fork

Bloque 5

upstream y copiar vs fork

Pregunta 25

Cuando clonas tu propio fork, ¿a qué apunta el remoto `origin` ?

- A) Al repositorio original del otro autor
- B) A tu fork
- C) A ningún sitio, hay que configurarlo a mano
- D) A `main` directamente

Pregunta 26

Si el repositorio original sigue avanzando después de que hiciste tu fork, ¿tu fork se actualiza solo?

- A) Sí, automáticamente
- B) No, hace falta traer esos cambios manualmente (por ejemplo, con un remoto `upstream`)
- C) Solo si haces `git clone` de nuevo cada día
- D) Solo si el propietario original te lo permite explícitamente cada vez

Pregunta 27

¿Qué comando añade el repositorio original como un segundo remoto en tu fork clonado?

- A) `git remote add upstream <url>`
- B) `git clone upstream <url>`
- C) `git branch upstream`
- D) `git push upstream`

Pregunta 28

Después de `git fetch upstream`, ¿qué comando usarías para traer esos cambios a tu rama `main`?

- A) `git remote remove upstream`
- B) `git merge upstream/main`
- C) `git push upstream main`
- D) `git clone upstream/main`

Pregunta 29

¿Qué diferencia hay entre un fork y simplemente copiar un repositorio (borrando la carpeta `.git` y subiéndolo a un repo propio)?

- A) Ninguna, son exactamente lo mismo
- B) El fork mantiene una relación con el repositorio original que permite contribuir; una copia sin `.git` no tiene ninguna relación
- C) La copia sin `.git` es más rápida de fusionar
- D) El fork no se puede clonar

Bloque 6

git remote

Pregunta 30

¿Para qué sirve el comando `git remote` ?

- A) Para gestionar las conexiones a repositorios remotos
- B) Para hacer commits
- C) Para crear ramas locales
- D) Para ver el historial de commits

Pregunta 31

¿Qué comando lista los repositorios remotos configurados, junto con sus URLs?

- A) `git remote list`
- B) `git remote -v`
- C) `git branch -r`
- D) `git log --remote`

Pregunta 32

¿Qué comando añade un nuevo repositorio remoto?

- A) `git remote add <nombre-remoto> <url>`
- B) `git remote new <url>`
- C) `git clone add <url>`
- D) `git push add <url>`

Pregunta 33

¿Qué nombre recibe, por convención, el repositorio remoto principal cuando solo tienes uno?

- A) `main`
- B) `master`
- C) `origin`
- D) `remote`

Pregunta 34

¿Qué comando elimina la referencia a un repositorio remoto?

- A) `git remote remove <nombre-remoto>`
- B) `git remote delete-all`
- C) `git branch -D <nombre-remoto>`
- D) `git push --remove <nombre-remoto>`

Pregunta 35

Si ejecutas `git remote -v` y no aparece nada, ¿qué significa?

- A) Que el repositorio está roto
- B) Que no hay ningún repositorio remoto configurado
- C) Que no tienes conexión a internet
- D) Que no has hecho ningún commit todavía

Bloque 7

`git fetch` vs `git pull`

Pregunta 36

¿Qué hace `git fetch` ?

- A) Descarga los últimos cambios del remoto, pero no los fusiona en tu rama local
- B) Descarga y fusiona automáticamente los cambios del remoto
- C) Sube tus cambios al remoto
- D) Borra los cambios locales

Pregunta 37

¿Qué hace `git pull` ?

- A) Solo descarga los cambios, sin tocar tu rama local
- B) Descarga los cambios del remoto y los fusiona automáticamente con tu rama local
- C) Sube tus commits al remoto
- D) Elimina una rama remota

Pregunta 38

¿Qué dos operaciones combina `git pull` ?

- A) `git add` y `git commit`
- B) `git fetch` y `git merge`
- C) `git clone` y `git push`
- D) `git branch` y `git checkout`

Pregunta 39

¿Cuándo es más útil usar `git fetch` en vez de `git pull` directamente?

- A) Nunca, `fetch` no sirve para nada
- B) Cuando quieres revisar los cambios del remoto antes de aplicarlos a tu rama local
- C) Cuando quieres borrar el historial remoto
- D) Solo la primera vez que clonas un repositorio

Pregunta 40

Después de un `git fetch`, ¿tus ficheros en el working directory han cambiado ya?

- A) Sí, siempre
- B) No, `fetch` no toca tu working directory, solo actualiza la información sobre el remoto
- C) Solo si usas `--force`
- D) Solo los ficheros ignorados

Pregunta 41

¿Qué comando usarías para traer los cambios del remoto `origin`, rama `main`, y fusionarlos directamente con tu rama actual?

- A) `git fetch origin main`
- B) `git pull origin main`
- C) `git push origin main`
- D) `git remote origin main`

Bloque 8

```
git push
```

Pregunta 42

¿Qué hace `git push` ?

- A) Descarga cambios del remoto
- B) Envía los commits de tu repositorio local hacia el repositorio remoto
- C) Crea un repositorio nuevo en GitHub
- D) Elimina commits del historial

Pregunta 43

¿Qué opción se usa la primera vez que subes una rama, para establecer la conexión de seguimiento con el remoto?

- A) `-u`
- B) `-f`
- C) `-d`
- D) `-m`

Pregunta 44

Después de hacer `git push -u origin main` una vez, ¿qué puedes hacer en los siguientes push a esa misma rama?

- A) Nada distinto, siempre hay que repetir `-u origin main`
- B) Simplemente `git push`, sin especificar remoto ni rama
- C) Ya no puedes volver a hacer push a esa rama
- D) Tienes que hacer `git clone` de nuevo

Pregunta 45

Si alguien ha subido cambios al remoto después de tu último `pull`, y tú intentas hacer `git push`, ¿qué es lo más probable que ocurra?

- A) Se sube sin problema y sobrescribe lo del remoto
- B) Git rechaza el push y te pide que actualices tu repositorio local primero
- C) Se crea automáticamente una rama nueva
- D) Git borra tus commits locales

Pregunta 46

¿Cuál de estos mensajes indica que un `git push` ha sido rechazado?

- A) "Fast-forward"
- B) "nothing to commit, working tree clean"
- C) "! [rejected] ... failed to push some refs"
- D) "Initialized empty Git repository"

Pregunta 47

Ante un push rechazado por cambios remotos que no tienes en local, ¿cuál es la solución correcta?

- A) Usar `git push --force` inmediatamente
- B) Hacer `git pull` primero y luego volver a intentar `git push`
- C) Borrar el repositorio remoto y crear uno nuevo
- D) Ignorarlo, no tiene solución

Pregunta 48

¿Por qué se desaconseja usar `git push --force` como primera solución ante un push rechazado?

- A) Porque no existe ese comando
- B) Porque puede sobrescribir y perder los commits que otros ya han subido al remoto
- C) Porque solo funciona en repositorios privados
- D) Porque tarda demasiado

Bloque 9

Ramas remotas

Pregunta 49

¿Qué son las ramas remotas en Git?

- A) Ramas que solo existen en tu ordenador
- B) Copias de las ramas locales que existen en un repositorio remoto como GitHub
- C) Un tipo de commit especial
- D) Ficheros de configuración

Pregunta 50

¿Qué nomenclatura suelen seguir las ramas remotas?

- A) `<nombre-rama>--<nombre-remoto>`
- B) `<nombre-remoto>/<nombre-rama>`
- C) `remote:<nombre-rama>`
- D) `<nombre-rama>@<nombre-remoto>`

Pregunta 51

¿Qué representa `origin/main` ?

- A) Una rama local llamada `origin`
- B) La rama `main` en el remoto `origin`
- C) Un commit concreto
- D) Un fichero de configuración

Pregunta 52

¿Qué comando lista las ramas remotas conocidas por tu repositorio local?

- A) `git branch -r`
- B) `git remote branches`
- C) `git log --remote`
- D) `git checkout -r`

Pregunta 53

Después de crear una rama local y hacer `git push -u origin feature-branch`, ¿qué aparece al ejecutar `git branch -r`?

- A) Nada, las ramas remotas no se pueden listar
- B) `origin/feature-branch`
- C) Solo `origin/main`
- D) `feature-branch` sin prefijo

Pregunta 54

¿Cómo traes a tu repositorio local los cambios que otro colaborador ha subido a `origin/feature-branch` ?

- A) `git branch -d origin/feature-branch`
- B) `git pull`
- C) `git remote remove origin`
- D) `git init`

Bloque 10

Pull Request

Pregunta 55

¿Qué es un Pull Request (PR)?

- A) Un comando de git para descargar cambios
- B) Una función de plataformas como GitHub para proponer cambios que luego son revisados antes de fusionarse
- C) Un tipo de rama remota
- D) Un fichero de configuración de GitHub

Pregunta 56

¿Qué principio establece que todos los miembros de un equipo son responsables de la calidad del trabajo colaborativo, según lo visto en clase?

- A) El principio de la rama única
- B) El principio de la responsabilidad compartida
- C) El principio del fast-forward
- D) El principio del fork obligatorio

Pregunta 57

Al crear un Pull Request en GitHub, ¿qué se recomienda incluir además del título?

- A) Nada más, con el título basta
- B) Una descripción clara de qué cambios se han hecho y por qué
- C) La contraseña de tu cuenta
- D) El historial completo de `git log`

Pregunta 58

¿Para qué sirve asignar revisores a un Pull Request?

- A) Para que revisen los cambios antes de que se fusionen
- B) Para que borren la rama automáticamente
- C) Para que el PR se cierre automáticamente
- D) No tiene ninguna función real

Pregunta 59

Un Pull Request, ¿se puede abrir tanto entre dos ramas del mismo repositorio como desde un fork hacia el repositorio original?

- A) No, solo entre ramas del mismo repositorio
- B) Sí, ambas situaciones son posibles
- C) No, solo desde un fork
- D) Solo si el repositorio es privado

Pregunta 60

Después de fusionar un Pull Request desde la interfaz web de GitHub, ¿qué comando necesitas ejecutar en tu terminal para tener ese cambio en tu `main` local?

- A) `git push`
- B) `git pull`
- C) `git remote add`
- D) `git clone`

Bloque 11

Modelos de ramas: GitFlow

Pregunta 61

¿Qué son los "modelos de ramas"?

- A) Comandos de git para crear ramas
- B) Enfoques estructurados para gestionar el flujo de trabajo de ramas en un proyecto
- C) Un tipo de conflicto de merge
- D) Extensiones de GitHub

Pregunta 62

Según lo visto en clase, ¿hay un modelo de ramas mejor que los demás en todos los casos?

- A) Sí, GitFlow es siempre el mejor
- B) No, depende del proyecto y de la fase en la que esté
- C) Sí, Trunk-Based Development es obligatorio en toda empresa
- D) No existen modelos de ramas, es un mito

Pregunta 63

En GitFlow, ¿qué dos ramas son las principales y permanentes?

- A) feature y hotfix
- B) main (o master) y develop
- C) origin y upstream
- D) release y trunk

Pregunta 64

En GitFlow, ¿a partir de qué rama se crean las *feature branches*?

- A) main
- B) develop
- C) release
- D) origin

Pregunta 65

En GitFlow, ¿para qué sirven las *hotfix branches*?

- A) Para preparar una nueva versión de `develop`
- B) Para arreglar errores urgentes directamente sobre `main`
- C) Para documentar el proyecto
- D) Para borrar ramas antiguas

Pregunta 66

¿Para qué tipo de proyectos está especialmente pensado GitFlow?

- A) Proyectos con despliegue continuo varias veces al día
- B) Desarrollos a largo plazo con versiones de software bien definidas
- C) Proyectos de una sola persona sin ramas
- D) Proyectos que nunca lanzan versiones

Bloque 12

GitLab Flow y Trunk-Based Development

Pregunta 67

¿En qué está especialmente orientado GitLab Flow?

- A) En mantener ramas de release abiertas durante años
- B) En el CI/CD, con una rama por cada entorno de desarrollo/producción
- C) En prohibir el uso de ramas
- D) En sustituir a GitHub por completo

Pregunta 68

En GitLab Flow, ¿qué ocurre con cada commit en una rama de entorno?

- A) Nada, no tiene ningún efecto
- B) Se genera un deploy en ese entorno
- C) Se borra automáticamente el commit anterior
- D) Se crea un fork automáticamente

Pregunta 69

¿Qué caracteriza al Trunk-Based Development?

- A) Cada desarrollador tiene su propia rama permanente de meses de duración
- B) Los desarrolladores colaboran en una única rama principal ("trunk", normalmente `main`)
- C) Prohíbe totalmente hacer commits en `main`
- D) Requiere obligatoriamente GitFlow como base

Pregunta 70

En Trunk-Based Development, si una funcionalidad no está lista para verse en producción, ¿qué se hace en vez de dejarla en una rama aparte durante mucho tiempo?

- A) Se elimina el código hasta que esté lista
- B) Se integra en el "trunk" oculta detrás de un *feature flag*
- C) Se sube directamente sin ningún control
- D) Se guarda solo en el ordenador del desarrollador, sin subirla nunca

Pregunta 71

¿Para qué tipo de proyectos es ideal el Trunk-Based Development, según lo visto en clase?

- A) Proyectos sin integración continua
- B) Proyectos que usan integración continua y despliegue continuo (CI/CD)
- C) Proyectos que solo lanzan una versión al año
- D) Proyectos sin ningún colaborador

Pregunta 72

¿Cuál de estos NO es un ejemplo mencionado en clase de proyecto que usa Trunk-Based Development?

- A) Kernel de Linux
- B) Chrome
- C) Facebook
- D) IntelliJ

Bloque 13

Preguntas de escenario

Pregunta 73

Quieres empezar a trabajar sobre un proyecto que ya existe en GitHub. ¿Qué comando usas para tener una copia completa en tu máquina, con `origin` ya configurado?

- A) `git init`
- B) `git clone <url>`
- C) `git remote add`
- D) `git branch -r`

Pregunta 74

Intentas hacer `git push` y GitHub te pide usuario y contraseña; introduces tu contraseña habitual y falla. ¿Cuál es la causa más probable?

- A) Has escrito mal tu nombre de usuario
- B) GitHub ya no acepta contraseña por HTTPS; necesitas un token o SSH
- C) Tu repositorio está corrupto
- D) No tienes conexión a internet

Pregunta 75

Quieres contribuir a un proyecto de código abierto que no es tuyo y en el que no tienes permisos de escritura. ¿Qué es lo primero que harías?

- A) Pedir la contraseña del propietario
- B) Hacer un fork del repositorio
- C) Borrar el repositorio y crear uno nuevo
- D) Hacer `git push --force` directamente

Pregunta 76

Llevas dos semanas trabajando en tu fork y el repositorio original ha recibido muchos commits nuevos que tú no tienes. ¿Qué necesitas para traerlos?

- A) Un remoto `upstream` configurado, y hacer `fetch` / `merge` desde él
- B) Nada, se actualiza solo
- C) Borrar tu fork y volver a hacerlo
- D) Cambiar el nombre de tu rama a `main`

Pregunta 77

Haces `git push` y recibes un mensaje `[rejected] ... failed to push some refs`.
¿Qué NO deberías hacer como primera reacción?

- A) `git pull` para traer los cambios que faltan
- B) `git push --force` para forzarlo
- C) Revisar qué cambios hay en el remoto que no tienes en local
- D) Resolver el posible conflicto tras el `pull`

Pregunta 78

Un compañero te dice que ha creado una rama, ha subido cambios y ha abierto un Pull Request hacia `main`. ¿Dónde se revisa y se fusiona ese Pull Request?

- A) Únicamente por terminal, con `git merge`
- B) Desde la interfaz web de GitHub (o la plataforma equivalente)
- C) No se puede fusionar, solo sirve para comentar
- D) Automáticamente, sin que nadie lo revise

Pregunta 79

Tu equipo despliega a producción varias veces al día directamente desde `main`, usando *feature flags* para ocultar funcionalidades a medio terminar. ¿Qué modelo de ramas están siguiendo?

- A) GitFlow
- B) Trunk-Based Development
- C) SVN Flow
- D) Ninguno, eso no es un modelo de ramas

Pregunta 80

Un proyecto necesita mantener varias versiones de software en paralelo (v1, v2, v3) con ramas de `release` y `hotfix` bien diferenciadas de `develop`. ¿Qué modelo encaja mejor?

- A) Trunk-Based Development
- B) GitFlow
- C) No hace falta ningún modelo
- D) GitLab Flow exclusivamente

Pregunta 81

Ejecutas `git remote -v` en un repositorio que acabas de clonar y ves solo `origin`. Quieres además poder traer cambios del repositorio del que hiciste fork. ¿Qué harías?

- A) `git remote add upstream <url_del_original>`
- B) `git branch -r upstream`
- C) `git clone upstream`
- D) `git push upstream`

¡Suerte con el repaso!